

AI1013_assignment_2_AI24BTECH11015_2

March 21, 2025

#

AI 1013 : Programming for AI

##

Assignment 2

0.0.1 2. Classification using the *k*-Nearest Neighbors Technique

2.1 The Abalone Dataset Importing the abalone dataset using the code snippet provided.

```
[1]: import pandas as pd
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/abalone/abalone.
      ↪data"
abalone = pd.read_csv(url, header=None)
abalone.head(3)
```

```
[1]:   0      1      2      3      4      5      6      7      8
0  M  0.455  0.365  0.095  0.5140  0.2245  0.1010  0.15  15
1  M  0.350  0.265  0.090  0.2255  0.0995  0.0485  0.07   7
2  F  0.530  0.420  0.135  0.6770  0.2565  0.1415  0.21   9
```

2.2 Data Pre-Processing and Cleaning To make the dataset easier to understand and work on, we start by renaming the columns to a more meaningful name.

```
[2]: column_names = ["Sex", "Length", "Diameter", "Height", "Whole weight", "Shucked_
      ↪weight", "Viscera weight", "Shell weight", "Rings"]
abalone.columns = column_names
abalone.head(3)
```

```
[2]:   Sex  Length  Diameter  Height  Whole weight  Shucked weight  Viscera weight  \
0   M    0.455    0.365    0.095    0.5140    0.2245    0.1010
1   M    0.350    0.265    0.090    0.2255    0.0995    0.0485
2   F    0.530    0.420    0.135    0.6770    0.2565    0.1415

      Shell weight  Rings
0              0.15    15
1              0.07     7
2              0.21     9
```

Since Sex of the abalone does not directly influence the age of the abalone, we can remove this parameter from our dataset.

```
[3]: abalone.drop(columns = "Sex", inplace = True)
      abalone.head(3)
```

```
[3]:   Length  Diameter  Height  Whole weight  Shucked weight  Viscera weight  \
0    0.455    0.365   0.095    0.5140    0.2245    0.1010
1    0.350    0.265   0.090    0.2255    0.0995    0.0485
2    0.530    0.420   0.135    0.6770    0.2565    0.1415

      Shell weight  Rings
0             0.15    15
1             0.07     7
2             0.21     9
```

2.3 Training and Test Data We will separate this data into two frames, one which contains all the features and the other which contains the target feature (Rings)

```
[4]: abalone_features = abalone.drop(columns = "Rings")
      abalone_rings = abalone["Rings"]
```

Now we will be splitting our dataset into two parts, one for training and the other for evaluation.

```
[5]: from sklearn.model_selection import train_test_split
      X_train, X_test, y_train, y_test = train_test_split(abalone_features.values,
      ↪ abalone_rings.values, test_size=0.3, random_state=333)
```

Now we have our training data and test data separated. The X values are large arrays of 7-D vectors. Each component of the vector is the numerical value of some feature of the abalone. The y values are arrays containing the number of rings each abalone has. We will have that the number of elements in X_train and y_train will be same as each abalone defined by a particular vector (in X_train) has a known number of rings (in y_train).

2.4 Implementing the k -Nearest Neighbors Algorithm We need to predict the age of a new abalone given by following attributes using the k -nearest neighbors method.

Variable	Value
Length	0.569552
Diameter	0.446407
Height	0.154437
Whole Weight	1.016849
Shucked Weight	0.439051
Viscera Weight	0.222526
Shell Weight	0.291208

Let's define this table as an array in our code.

```
[6]: import numpy as np
new_abalone = np.array([0.569552, 0.446407, 0.154437, 1.016849, 0.439051, 0.
↪ 222526, 0.291208])
```

Calculating euclidean distance between each row of `X_train` and the `new_abalone`

```
[7]: distances = []
for training_point in X_train:
    distances.append(np.linalg.norm(training_point - new_abalone))
distances = np.array(distances)
```

Sorting the distances values and finding the indices of the 3 nearest neighbors of the `new_abalone`

```
[8]: #getting sorted list of indices (in ascending order)
indices = distances.argsort()

#keeping only the first k elements (here k=3)
indices = indices[:3]
print(f"The 3 nearest neighbors have indices {indices[0]}, {indices[1]} and ↵
↪ {indices[2]}".)

#putting the corresponding y_values (age / no. of rings) in an array
k_nearest_ages = np.array([y_train[i] for i in indices])
print(f"The corresponding number of rings are {k_nearest_ages[0]}, ↵
↪ {k_nearest_ages[1]} and {k_nearest_ages[2]}".)
```

The 3 nearest neighbors have indices 1461, 774 and 1673.

The corresponding number of rings are 10, 11 and 11.

Now to calculate mode as per our requirements, we will define a function.

```
[9]: from statistics import multimode

def find_mode(numbers):
    #getting an array of mode(s)
    mode = multimode(numbers)

    #Return the mode
    #return randmoly if multiple modes found
    np.random.seed(123)
    return np.random.choice(mode)
```

Now calculating the mode of the data.

```
[10]: predicted_age = find_mode(k_nearest_ages)
print(f"The predicted age of the new_abalone is {predicted_age}")
```

The predicted age of the `new_abalone` is 11

Now, as directed we will predict the age of the abalone datapoints in the `X_test` and calculate the MSE between predicted value and the actual value of the age. We will ease our work by defining a

function which will predict the age.

```
[11]: def predict_age(k, X_train, y_train, abalone_point):  
    #Calculating euclidean distances  
    distances = np.linalg.norm(X_train - abalone_point, axis=1)  
  
    #getting the k nearest neighbours  
    indices = np.argsort(distances)  
    indices = indices[:k]  
  
    #fetching the corresponding y values in an array  
    k_nearest_ages = np.array([y_train[i] for i in indices])  
  
    #returning the mode of the k nearest ages as the predicted age  
    return find_mode(k_nearest_ages)
```

Now using this function to predict the ages and store them in array.

```
[12]: # predicted_ages_test = np.array([predict_age(3, X_train, y_train, data_point)␣  
    ↪for data_point in X_test])  
predicted_ages_test = []  
for data_point in X_test:  
    predicted_ages_test.append(predict_age(3,X_train, y_train, data_point))  
predicted_ages_test = np.array(predicted_ages_test)
```

Now lets define a function to calculate the MSE and return it.

```
[13]: def calculate_MSE(predicted, actual):  
    squared_sum = 0  
  
    for i in range(len(predicted)):  
        squared_sum += (predicted[i] - actual[i])**2  
  
    return squared_sum/len(predicted)
```

Calculating the MSE for $k = 3$ and printing it.

```
[14]: MSE = calculate_MSE(predicted_ages_test, y_test)  
print(f"The MSE for k=3 is {MSE}")
```

The MSE for k=3 is 7.515948963317384

2.5 Tuning k to Achieve Optimal Performance.

```
[15]: #generating array of k values  
k_values = np.arange(1,51)  
  
    #Calculating MSE value for all the k values  
mse_values = []  
for i in range(len(k_values)):
```

```

    predicted_ages_test = np.array([predict_age(k_values[i], X_train, y_train,
↪data_point) for data_point in X_test])
    mse_values.append(calculate_MSE(predicted_ages_test, y_test))

mse_values = np.array(mse_values)

```

Lets now plot mse_values vs k_values.

```

[16]: #finding the optimal k-value for which minimum MSE occurs.
    optimal_k = mse_values.argsort()[0] + 1
    min_mse = mse_values[optimal_k - 1]
    print("The Optimal value of k =", optimal_k)
    print("MSE for optimal value of k =",min_mse)

    import matplotlib.pyplot as plt
    plt.figure(figsize=(10, 6))

    #plotting the mse vs k
    plt.plot(k_values, mse_values, "o-", label="MSE vs k-value", color="royalblue")

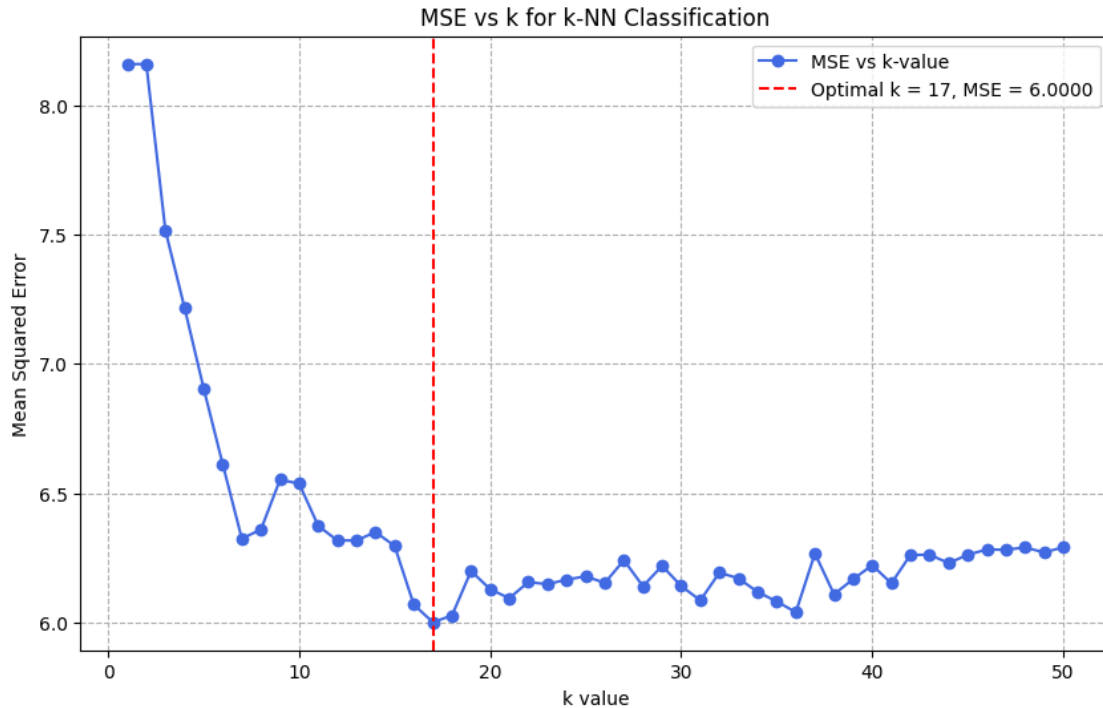
    #plotting the optimal k value
    plt.axvline(x=optimal_k, color='r', linestyle='--', label=f'Optimal k =
↪{optimal_k}, MSE = {min_mse:.4f}')

    plt.legend()
    plt.grid(linestyle="--")
    plt.xlabel("k value")
    plt.ylabel("Mean Squared Error")
    plt.title("MSE vs k for k-NN Classification")
    plt.show()

```

The Optimal value of k = 17

MSE for optimal value of k = 6.0



The optimal value of k for which MSE is minimum is $k = 17$. For better results, let's implement k-fold cross-validation to find the optimal value of k .

K-Fold Cross-Validation For K -fold cross-validation, we will take the value $K = 5$ so that our test data is 20% of the total data.

```
[17]: num_folds = 5

np.random.seed(15)

#randomly creating folds
indices = np.random.permutation(len(abalone_features))
folds = np.array_split(indices, num_folds)

#making the folds
feature_folds = [abalone_features.iloc[fold] for fold in folds]
rings_folds = [abalone_rings.iloc[fold] for fold in folds]
```

Now we have our folds, let's calculate the mean MSE for $k = 1$ to $k = 30$ to compare them.

```
[18]: #generating array of k-values
k_values = np.arange(1,31)

mean_mse_values = [0 for _ in k_values]
#Calculating mean MSE for all k values using k fold
```

```

for k in k_values:
    for i in range(num_folds):
        #getting the testing data
        X_test, y_test = feature_folds[i].to_numpy(), rings_folds[i].to_numpy()

        #getting the training data
        X_train = pd.concat([feature_folds[j] for j in range(num_folds) if j!
        =>i]).to_numpy()
        y_train = pd.concat([rings_folds[j] for j in range(num_folds) if j!=i]).
        =>to_numpy()

        #Calculating mse and adding to array
        predicted_ages_test = np.array([predict_age(k, X_train, y_train,
        =>data_point) for data_point in X_test])
        mean_mse_values[k-1] += (calculate_MSE(predicted_ages_test, y_test))

mean_mse_values = np.array(mean_mse_values)
mean_mse_values /= num_folds

```

Now we have our mean mse values for all values of K , lets plot it and get the optimal value of k

```

[19]: optimal_k_kfolds = mean_mse_values.argsort()[0] + 1
min_mse_kfolds = mean_mse_values[optimal_k_kfolds - 1]
print("The Optimal value of k =", optimal_k_kfolds)
print("MSE for optimal value of k =",min_mse_kfolds)

plt.figure(figsize=(10, 6))

#plotting the mse vs k
plt.plot(k_values, mean_mse_values, "o-", label="MSE vs k-value (Via K-Folds_
=>method)", color="royalblue")

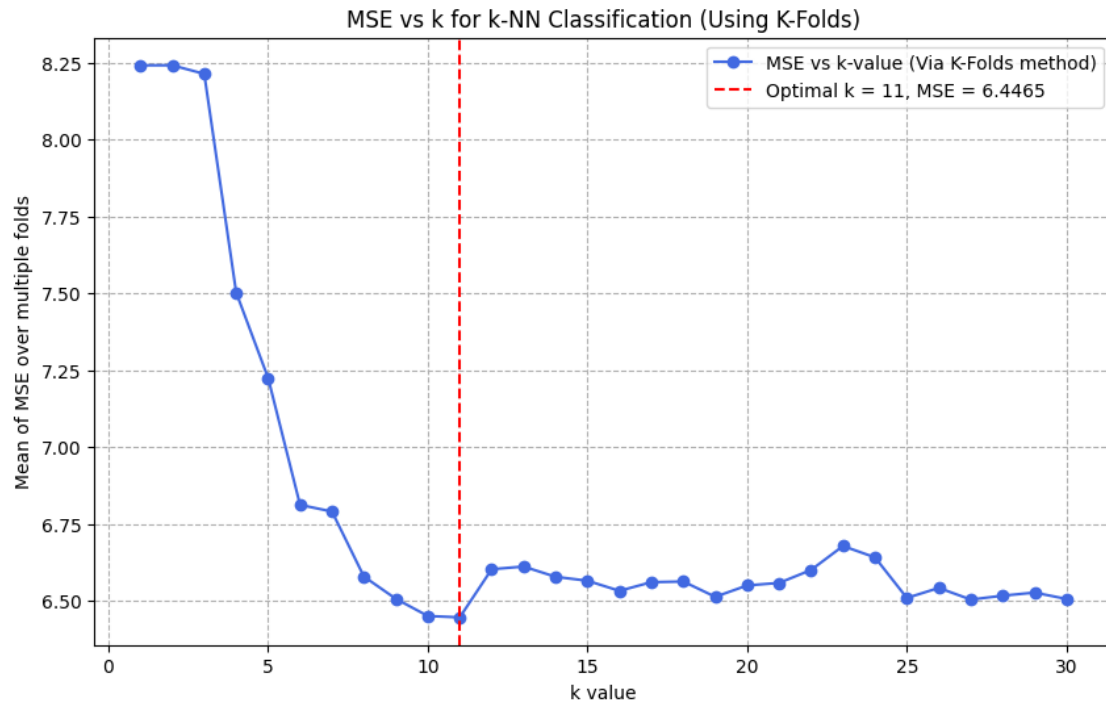
#plotting the optimal k value
plt.axvline(x=optimal_k_kfolds, color='r', linestyle='--', label=f'Optimal k =_
=>{optimal_k_kfolds}, MSE = {min_mse_kfolds:.4f}')

plt.legend()
plt.grid(linestyle="--")
plt.xlabel("k value")
plt.ylabel("Mean of MSE over multiple folds")
plt.title("MSE vs k for k-NN Classification (Using K-Folds)")
plt.show()

```

The Optimal value of $k = 11$

MSE for optimal value of $k = 6.446540411998969$



It is clear from the graph that $k = 11$ is a better choice for k as compared to $k = 17$ obtained by a single test-train split